Módulo 2

Repositorio y WorkFlow

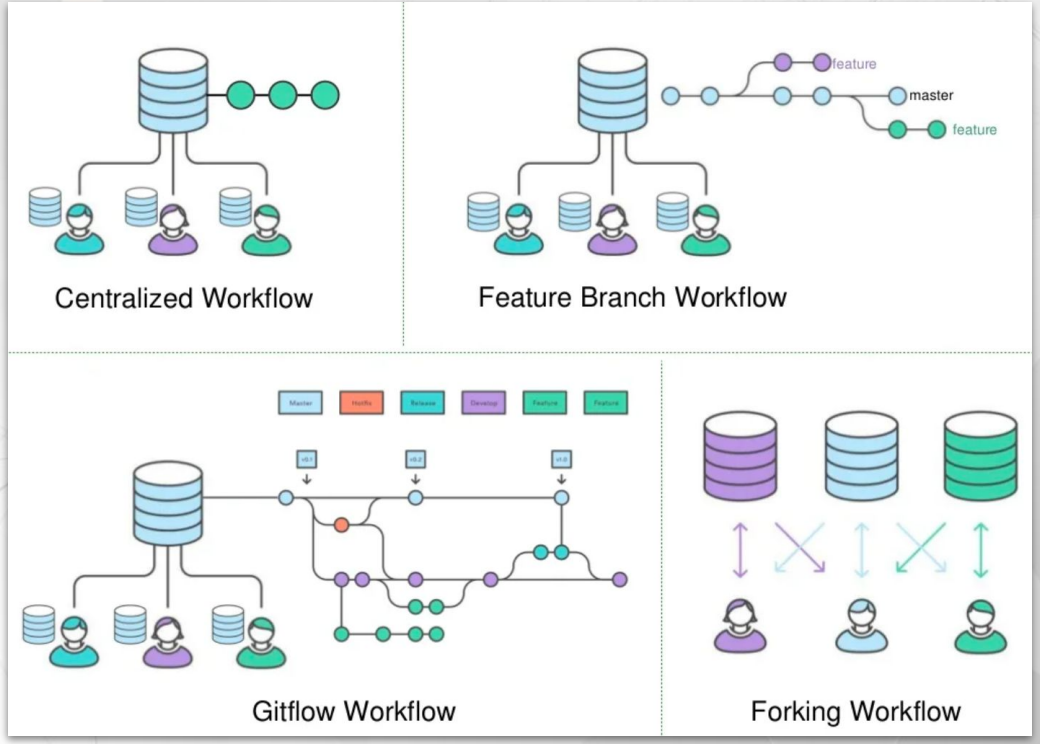
Control de versiones

- **Versión**: Un estado específico de un archivo o documento en un momento dado.
- **Control de versiones**: El proceso de gestión de diferentes versiones de un archivo o conjunto de archivos.
- **Historial de revisiones**: Un registro de todos los cambios realizados en un archivo o documento, incluyendo quién los hizo y cuándo.
- **Ramas:** Permiten crear diferentes líneas de desarrollo de un proyecto, lo que facilita la experimentación y el trabajo en paralelo sin afectar la versión principal.

Un **Sistema de control de versiones** es aquel que permite gestionar las diferentes revisiones o versiones de un documento, código o archivo a lo largo del tiempo. Permite rastrear los cambios realizados, identificar quién los hizo y cuándo, y facilita la recuperación de versiones anteriores. Es una práctica fundamental en el desarrollo de software, pero también útil en otros campos donde se trabaja con documentos y archivos que evolucionan.

Control de versiones

El control de versiones, también conocido como "control de código fuente", es la práctica de rastrear y gestionar los cambios en el código de software. Los sistemas de control de versiones son herramientas de software que ayudan a los equipos de software a gestionar los cambios en el código fuente a lo largo del tiempo.



Repositorio

Los repositorios y archivos digitales de acceso abierto son espacios virtuales, con soporte de base de datos, en los que se puede depositar documentación científica de todo tipo y en todos los formatos posibles.

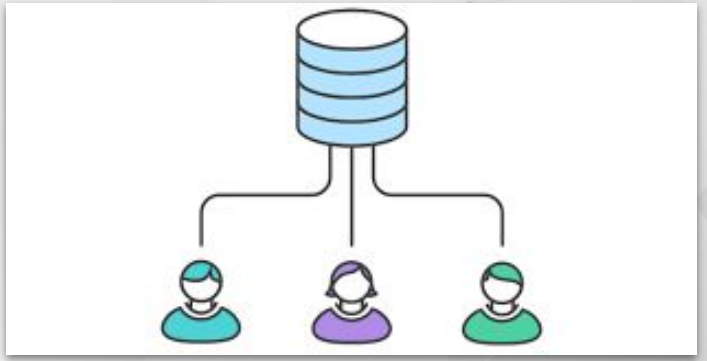
En Programación, un **repositorio** es un espacio donde se almacenan y gestionan los archivos de un proyecto de software, generalmente utilizando un sistema de control de versiones como Git. Este permite a los desarrolladores colaborar en el código fuente, mantener un historial de cambios, y coordinar el desarrollo. Pueden ser locales, almacenados en la máquina del desarrollador, o remotos, alojados en plataformas como GitHub, GitLab o Bitbucket.

El **flujo de trabajo** se refiere a la serie de pasos o actividades que un desarrollador sigue para completar una tarea específica o llevar a cabo un proceso en el desarrollo de software.

Repositorio Git

En Git un repositorio es un proyecto de software controlado por este sistema de control de versiones. Físicamente no es más que una carpeta del sistema de archivos con código fuente de una aplicación.

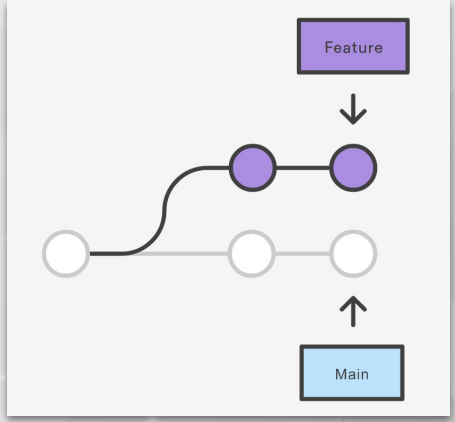
Pasar de un sistema de control de versiones centralizado a Git cambia la forma en que tu equipo de desarrollo crea software. Y si tu empresa depende de su software para aplicaciones críticas, la modificación del flujo de trabajo de desarrollo afecta a toda la empresa.



Flujo de trabajo

Una de las mayores ventajas de Git son sus capacidades de ramificación. A diferencia de los sistemas de control de versiones centralizados, las ramas de Git son baratas y fáciles de fusionar. Esto facilita el flujo de trabajo de ramas de función tan popular entre muchos usuarios de Git.

Las ramas de función proporcionan un entorno aislado para cada cambio en tu código base. Cuando un desarrollador quiere empezar a trabajar en algo, sin importar lo grande o pequeño sea, crea una nueva rama. Esto garantiza que la rama principal siempre contenga código de calidad para producción.



El uso de ramas de función no solo es más fiable que editar directamente el código de producción, sino que además proporciona ventajas organizativas. Te permite representar el trabajo de desarrollo con la misma granularidad que tu backlog ágil.

Sitios y Herramientas

Silicon Misiones

- Git
- GitHub
- GitHub Desktop

Configuraciones iniciales

git init: creará un nuevo repositorio local GIT.

- `git init`
- `git init [nombre del proyecto]`

git config: puede ser usado para establecer una configuración específica de usuario, como el email, nombre de usuario y tipo de formato, etc. Por ejemplo:

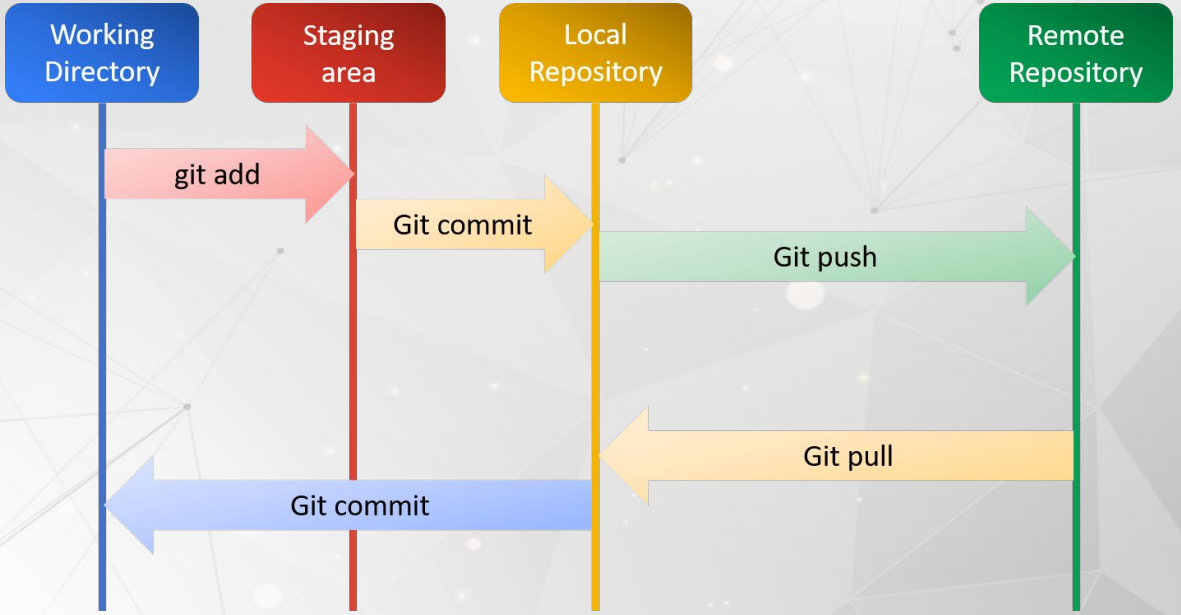
- `git config --global user.email tuemail@ejemplo.com`
- `git config --local user.email tuemail@ejemplo.com`

La opción -global le dice a GIT que vas a usar ese correo electrónico para todos los repositorios locales.

Comandos GIT básicos

Comandos más frecuentes:

- git status
- git clone
- git checkout
- git add
- git commit
- git push
- git pull
- git merge



Git status

Nos muestra los archivos modificados en “staging” y “no staging” después del último commit

Syntax of git status

```
git status
```

Es decir, muestra la lista de los archivos que se han cambiado junto con los archivos que están por ser preparados o confirmados.

Git clone

Descarga (copia) un repositorio a nuestra computadora

Syntax of git clone

```
git clone < Repository URL>
```

Si el repositorio está en un servidor remoto, usa:

```
git clone nombredeusuario@host:/path/to/repository
```

Git add

Agrega archivos modificados a área de trabajo local “staging”, la llamamos área de preparación

Syntax to add all modified file

```
git add .
```

Syntax to add single modified file

```
git add <File Name with full path>
```

Por ej., para indexar el archivo temp.txt escribimos: `git add temp.txt`

Git commit

Agrega los cambios del área “staging” al “repositorio local”. creará una instantánea de los cambios y la guardará en el directorio git.

Syntax of git commit

```
git commit -m "add commit message here"
```

Ten en cuenta que los cambios confirmados no llegarán al repositorio remoto.

Git push

Sube los cambios desde “repositorio local” al “repositorio remoto”. Se usa para enviar confirmaciones locales a la rama maestra o alguna rama a elección.

```
git push  origin <master>
```

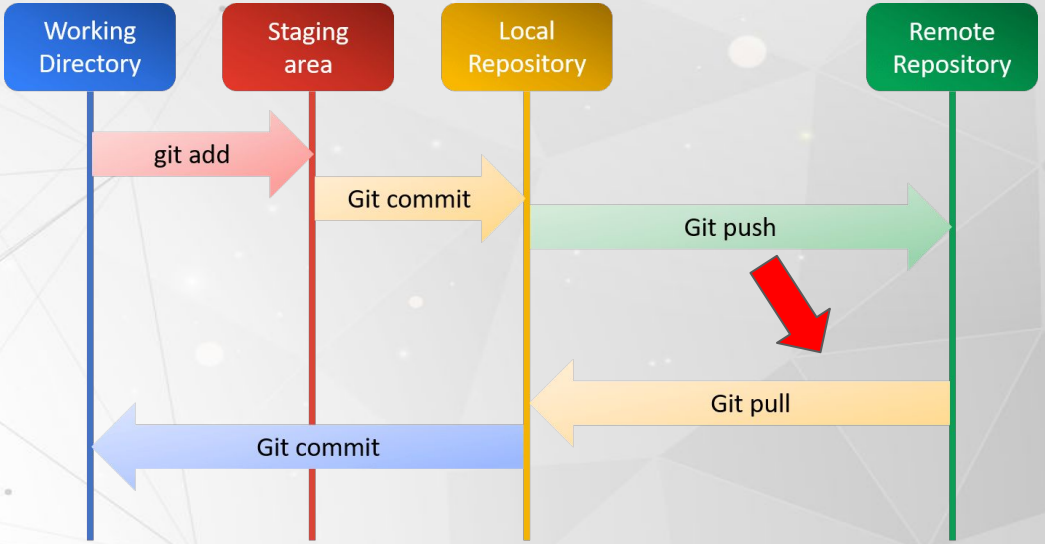
Reemplaza <master> con la rama en la que quieres enviar los cambios cuando no quieras enviarlos a la rama maestra.

Git pull

Descarga cambios desde “repositorio remoto” al “repositorio local”.
Usado para recuperar cambios realizados por los demás desarrolladores, fusiona todos los cambios que se han hecho en el repositorio remoto con el directorio de trabajo local.

Syntax of git pull

```
git pull
```

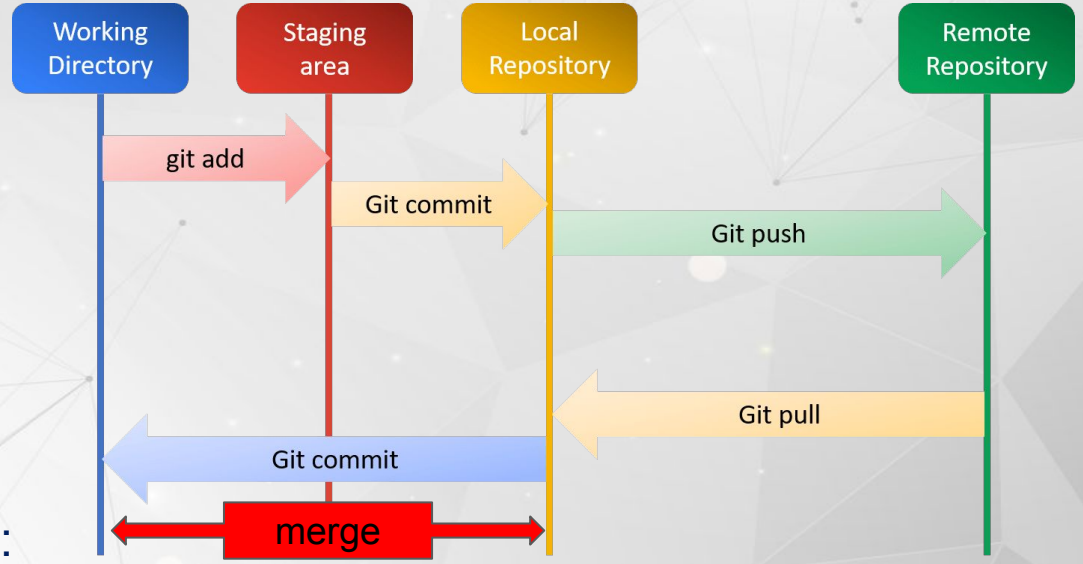


Git merge

Une cambios entre “repositorio local” y “working directory”

Syntax of git merge

```
git merge <commit id>
```



También se usa para fusionar una rama con otra rama activa:
`git merge <branch-name>`

Git checkout

Nos permite cambiarnos de rama del código.

Syntax of git checkout

```
git checkout <Branch Name>
```

También crea ramas y te ayuda a navegar entre ellas. Por ejemplo, el siguiente comando crea una nueva y automáticamente se cambia a ella:

```
command git checkout -b <branch-name>
```

Ramas en GIT (branch)

Una rama en Git es simplemente un puntero a un commit específico. Sirve para desarrollar funcionalidades, corregir errores o experimentar, sin afectar la rama principal (main o master). Al crear una rama, estás creando una **copia del estado actual del proyecto** desde donde seguirás trabajando.

```
git branch                      # Ver las ramas existentes
git branch nombre_rama          # Crear una nueva rama
git switch nombre_rama          # Cambiarse a otra rama
git switch -c correccion-bug    # Crear y cambiarse en un solo paso
git checkout nombre_rama        # Cambiarse a otra rama   (clásico)
git checkout -b correccion-bug  # Crear y cambiarse       (clásico)
git switch main                 # Volver a la rama principal
git merge nombre_rama           # Fusionar una rama a otra (merge)
git branch -d nombre_rama       # Eliminar una rama (ya fusionada)
git branch -m nuevo_nombre      # Cambiar nombre de la rama actual
```

Ramas en GIT - Ejemplo

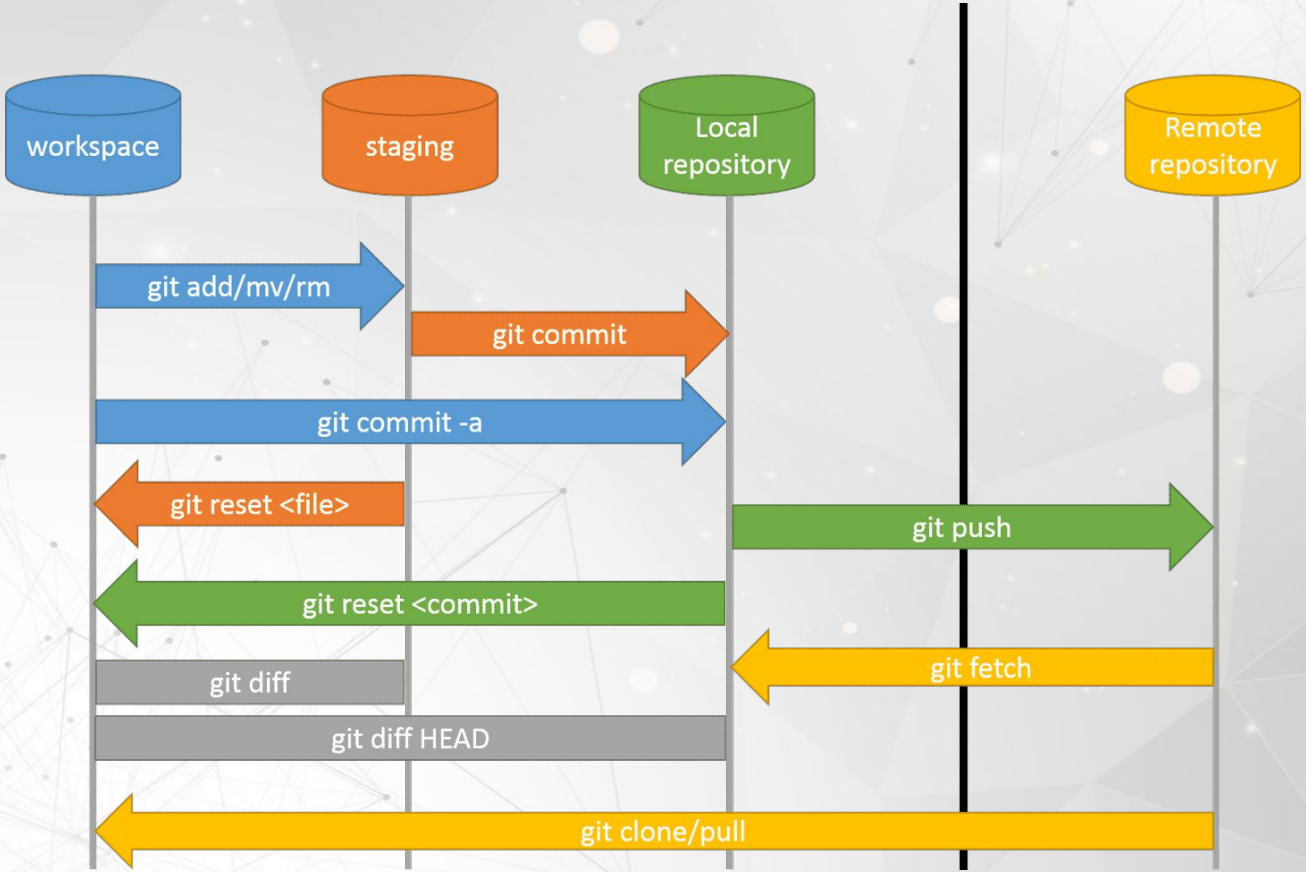
Supongamos que estás en main y querés trabajar en una nueva función:

```
git switch -c agregar-login      # Crea rama y cambia a ella
                                 # Editás archivos, hacés commits...
git add .
git commit -m "Implementé login básico"

git switch main                  # Volvés a main
git merge agregar-login          # Fusionás los cambios
git branch -d agregar-login      # Eliminar la rama (ya fusionada)

// git branch -D agregar-login   # Forzar borrado
```

Otros comandos GIT



Comandos útiles

Configuración

```
git config --global user.name "Tu Nombre"
git config --global user.email "tuemail@example.com"
git config --global core.editor code # o vim, nano, etc.

git config --global --list       # Ver configuración global
git config --local --list        # Ver configuración local del repo
git config --system --list       # Ver configuración del sistema
```

Inicialización y estado

```
git init                         # Inicializar repositorio
git status                       # Ver estado del repositorio
git add archivo.txt              # Agregar archivo al staging
git add .                        # Agregar todos los archivos
git commit -m "Mensaje"          # Crear commit con mensaje
```

Comandos útiles

Silicon Misiones

Logs y visualización del historial

```
git log                          # Ver historial completo
git log --oneline                # Historial compacto
git log --graph                  # Ver árbol de commits
git log --stat                   # Ver archivos modificados en cada commit
git log -p                       # Ver diferencias línea por línea
git log autor@example.com        # Filtrar por autor
git show <commit>                # Ver detalles de un commit
```

Trabajo Remoto

```
git remote add origin https://github.com/usuario/repositorio.git
git remote -v                    # Ver repos remotos
git fetch                        # Traer datos del remoto sin mezclar
git pull                         # Traer y fusionar datos
git push origin main             # Subir cambios
```

Comandos útiles

Silicon Misiones

Cambios de ramas y commits

```
git branch                    # Listar ramas
git branch nueva-rama         # Crear nueva rama
git switch nombre-rama        # Cambiar de rama (nuevo)
git checkout nombre-rama      # Cambiar de rama (antiguo)

git switch -c nueva-rama      # Crear y moverse a una rama
git checkout -b nueva-rama    # Equivalente al anterior

git switch -                  # Volver a la rama anterior
git restore archivo.txt       # Descartar cambios en el archivo
```

Moverse entre commits

```
git checkout <commit>         # Ir a un commit específico (modo detached HEAD)
git switch --detach <commit>  # Alternativa moderna a checkout
git log --oneline             # Ver los hashes cortos de los commits
```

Subir un proyecto local a GitHub

1. Crear un repositorio en GitHub
2. Inicializar Git en tu proyecto local y configurar variables globales
   a. `cd /ruta/del/proyecto`
   b. `git init`
   c. `git add .`
   d. `git commit -m "Primer commit del proyecto"`
3. Vincular el repositorio local con GitHub
   a. `git branch -m master main` *(opcional)*
   b. `git remote add origin https://github.com/tu-usuario/nombre-repo.git`
4. Subir los archivos al repositorio remoto
   a. `git push -u origin master` / `git push -u origin main`

Bajar un proyecto existente

1. Clonar el proyecto del repositorio en GitHub correspondiente.
   a. `cd /ruta/del/proyecto`
   b. `git clone https://github.com/otro-usuario/nombre-repo.git`
2. Idealmente empezar a trabajar y subir tus cambios a una rama distinta
   a. `cd nombre-repo`
   b. `git checkout -b mi-rama`
   c. `git add .`
   d. `git commit -m "Descripción de mis cambios"`
   e. `git push origin mi-rama`

git fetch vs git pull

git fetch descarga todos los cambios del repositorio remoto a tu repositorio local, pero no actualiza tu rama de trabajo actual automáticamente. Los cambios se almacenan en una rama de seguimiento remoto (origin/main, por ejemplo) sin alterar tus archivos locales.

- Se utiliza cuando quieres ver qué ha cambiado en el repositorio remoto antes de combinar esos cambios con tu trabajo local. Esto te permite revisar los cambios antes de hacer un merge manualmente.

```
git fetch origin
```

git pull es una combinación de git fetch y git merge. Descarga los cambios del repositorio remoto y los fusiona automáticamente con tu rama de trabajo actual.

- Se usa cuando confías en que los cambios remotos son seguros para ser fusionados con tu trabajo local y no necesitas revisarlos antes de hacerlo.

```
git pull origin main
```

git pull --merge y git pull --rebase

Después de descargar los cambios remotos con fetch, **`git pull --merge`** realiza un merge (fusión) automático de esos cambios con tu rama local. Si hay diferencias, se crea un nuevo commit de fusión. Es la opción predeterminada y es adecuada cuando quieres un historial de commits claro que muestre todas las fusiones de cambios en un solo lugar. Mantiene un historial explícito de cómo y cuándo se fusionaron las ramas. Ideal para equipos grandes y proyectos con ramas que se fusionan frecuentemente.

En lugar de fusionar los cambios, **`git pull --rebase`** aplica tus commits locales encima de los commits traídos del remoto, uno por uno. Mueve tus cambios locales al "final" del historial para que parezca que siempre se trabajó sobre la versión más actualizada. Se utiliza cuando prefieres un historial de commits más lineal, sin los commits adicionales de merge. Esto es ideal para mantener el historial limpio y fácil de leer.

Configurar repositorio remoto

Cuando Git te dice que ya tienes un repositorio remoto configurado, es porque ya existe una referencia a un repositorio remoto bajo el nombre origin. Para resolver esto, puedes eliminar la referencia actual y luego volver a añadir la dirección correcta.

- Ver las URLs de los remotos configurados para asegurarte de que origin está apuntando a una URL que deseas eliminar o cambiar

  ```
  git remote -v
  ```

- Para eliminar el remoto origin actual, usa:

  ```
  git remote remove origin
  ```

- Una vez eliminado el remoto, puedes agregar el nuevo remoto con la URL correcta

  ```
  git remote add origin https://github.com/user/project.git
  ```

- Ahora que el remoto está configurado correctamente, puedes proceder con:

  ```
  git push -u origin main
  ```

Pull Request

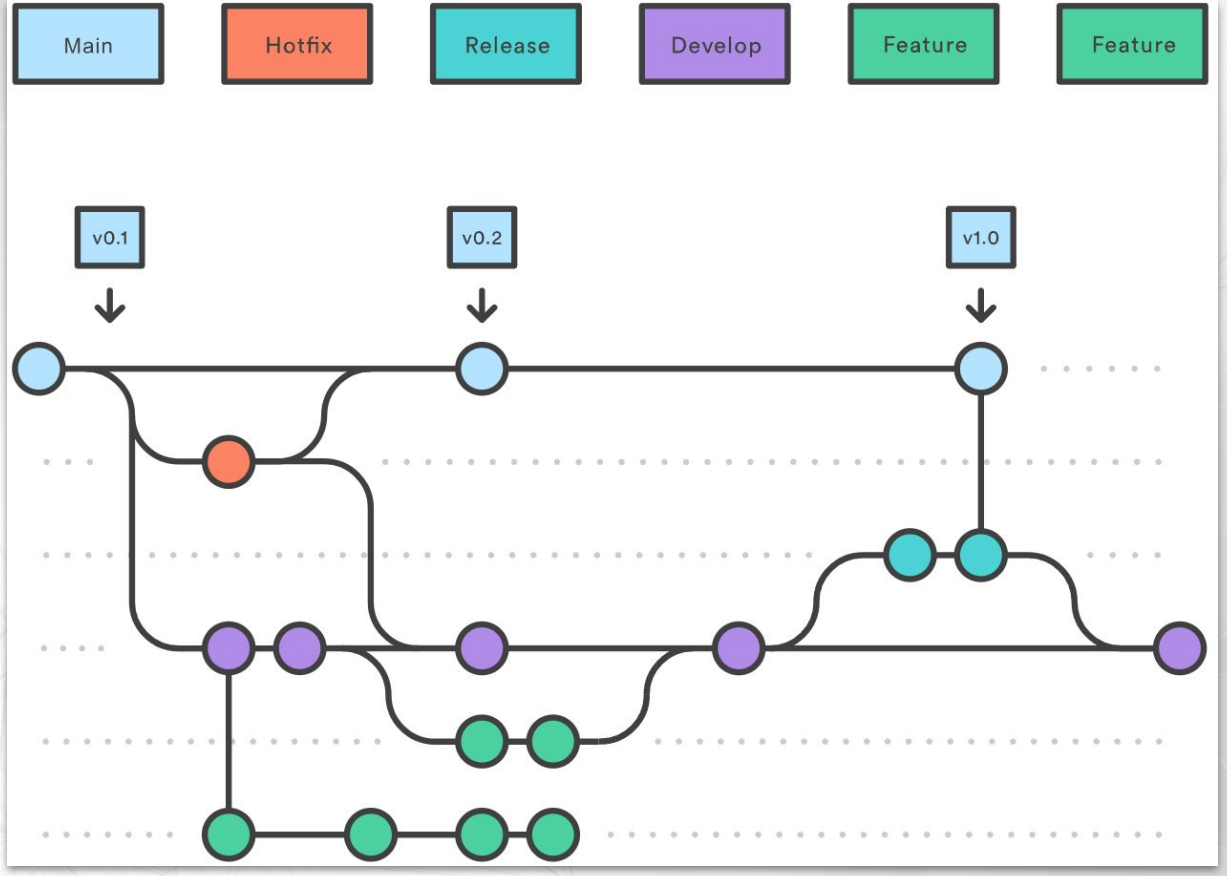
En GitHub, un "Pull Request" (solicitud de extracción o solicitud de cambios) es una propuesta para que un usuario incorpore sus cambios (commits) de su propia rama a la rama principal (o una rama de destino) de un proyecto. Es una forma de colaboración y revisión de código, donde el usuario que realiza el Pull Request solicita que otro usuario (o equipo) revise, apruebe y luego fusionar sus cambios.

En resumen, un Pull Request es una forma de proponer y discutir cambios a un proyecto de código, antes de que esos cambios se integren en la rama principal.

Issues

En GitHub, las "issues" (también conocidas como "problemas" o "incidencias") son una herramienta para planificar, rastrear y gestionar el trabajo dentro de un repositorio. Son una forma de registrar ideas, preguntas, errores, tareas o solicitudes de características, y permiten la colaboración y discusión en torno a ellas.

Útiles para

- Reportar un error de código en el repositorio.
- Solicitar una nueva función para la aplicación.
- Preguntar sobre la implementación de una característica.
- Organizar las tareas para la próxima versión del proyecto.
- Realizar un seguimiento de las tareas pendientes.

Gitflow